

Generative UI for Agent-Driven Interfaces

A practical guide to designing interfaces where agents don't just answer — they *act*, *plan*, and *shape* the UI itself. Built for product designers, UX engineers, and technical PMs working at the frontier of agent orchestration.

DESIGN × ENGINEERING

CHAINLIT · MULTI-AGENT · UX



What Is "Generative UI" in the Context of Agents?

Generative UI is a pattern where agents **emit structured descriptions of UI elements** — cards, panels, controls — that the frontend renders dynamically at runtime. Instead of replying with only raw text, the agent can suggest or directly control pieces of the interface: an order card, a product grid, a mini confirmation form.

This is fundamentally different from a fancy chat window. The UI becomes part of the agent's **action space** — a surface the agent can write to, not just speak through.

Reflection

The UI reflects *what the agent is doing* and what the user can do next — not just what it said.

Interaction

The agent can *propose interactions*, not just answers — turning passive output into active collaboration.



Chapter 1 · Foundations

What Makes a Good Agent UI Beyond Plain Chat?

A flat chat log isn't enough for agentic systems. Two principles must underpin every design decision:

Transparency

Show which tools, data sources, or agents are involved — at least at a high level. Answer the user's silent questions: *"What is it doing?"* and *"Why did it say that?"* Transparency is the foundation of trust.

Control

Give users a way to steer or interrupt actions — buttons, quick replies, confirmation prompts — not just free-text input. When actions carry costs or side-effects, users must feel ownership of outcomes, not passengers in an opaque process.

 Together, transparency and control transform an agent UI from a black box into a **collaborative workspace** users can trust and steer.

How Should the UI Represent Multi-Agent Behavior?

In a system with an Orchestrator plus specialist agents, the UI must surface structure without drowning users in internals.

What to Make Visible

- Which **kind of agent** is currently active — support-oriented vs. recommendation-oriented — using lightweight labels or tone cues
- Key **handoff moments** where the Orchestrator delegates to a specialist, surfaced as subtle section breaks, tags, or timeline markers

Why It Matters

Users get intuitive cues explaining *why* a response looks and sounds the way it does — different tool sets, different tones, different goals.

Developers and reviewers can glance at the UI and mentally reconstruct the same flow they'd expect in execution traces — making debugging and QA dramatically faster.



Orchestrator

Specialist

UI Handoff

Chapter 2 · Multi-Agent Design

How Can Generative UI Show Tool and Data Usage?

When an agent calls tools or RAG pipelines, unstructured paragraphs aren't enough. The interface should surface results in ways users and engineers can both act on.



Structured Result Cards

Render orders, products, tickets, and records as **typed UI cards** with clearly labeled fields — not unstructured prose. Users scan; they don't read paragraphs to find an order status.



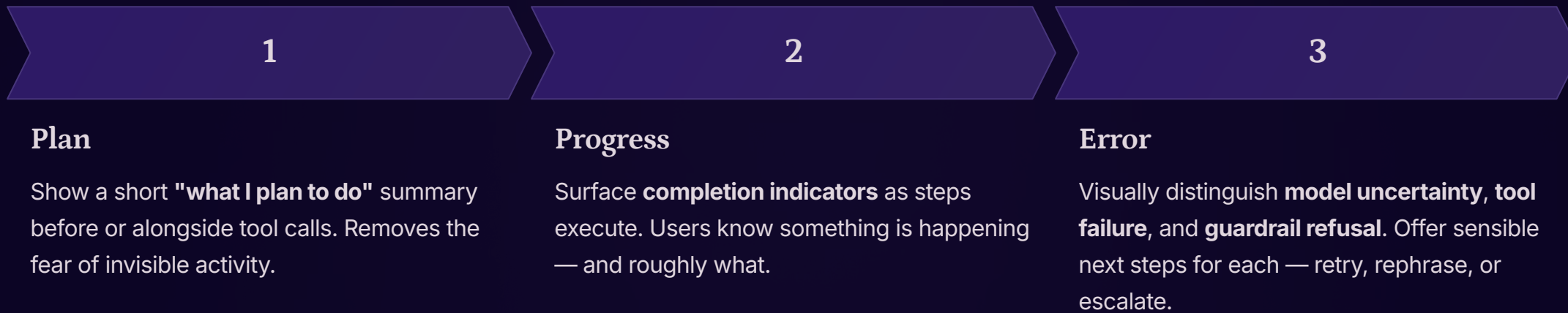
Source Badges & Freshness Indicators

Lightweight badges or icons show **where data came from** — live system, historical memory, retrieved document — and whether the result is fresh or cached. Engineers can map these directly to tool schemas.

- ✓ When UI components align with tool schemas, keeping the frontend and backend in sync becomes a **structural property**, not a documentation problem.

How Should the UI Handle Planning, Progress, and Errors?

Agents plan, act, and sometimes fail. The UI must reflect each phase of that lifecycle — reducing anxiety, building confidence, and enabling recovery.



Clear error states help users **recover**, not guess. A plan makes users feel like partners. Progress removes the anxiety of "is something happening without my knowledge?" — a question that erodes trust in seconds.

How Does Generative UI Support Multi-Turn Journeys?



Most real tasks span multiple steps, agents, and decisions. Thinking about UI over a **whole journey** — not just individual turns — unlocks a fundamentally better user experience.

1 Grouped Flows

Cluster messages, cards, and decisions into **named flows** (e.g., "Order Issue + Follow-Up Recommendation") instead of a flat, undifferentiated list.

2 Activity Summary

Surface a compact summary — which agents were involved, which tools were used — without forcing users to scroll through every intermediate step.

3 Revisit & Adjust

Creates a natural foundation for "**undo**", "**re-run this step**", and "**try a different option**" — features users expect in high-stakes workflows.

The Agent–UI Contract: Framework vs. Agent Responsibilities

A clean separation of concerns between your UI framework and your agents is the key to a maintainable, evolvable system.

UI Framework (e.g., Chainlit)

Responsible for **rendering structured components** — cards, panels, buttons, traces. It consumes the agent's output and presents it faithfully. It handles layout, interaction callbacks, and progressive disclosure of metadata.

Agent Layer

Responsible for **emitting clear, structured outputs** that describe what to render and why — e.g., *"here is an order card"* plus summary text. The agent defines the UI contract; the framework fulfills it.

- ❏ As long as structured outputs stay stable, you can evolve agent behaviors **without rewriting the UI** — and switch frameworks without renegotiating agent logic. The contract is the interface.

How Do You Avoid Overwhelming Users With Internals?

The Solution: Progressive Disclosure

Keep the main conversation clean and readable. Tuck detailed traces, tool parameters, and execution logs **behind expanders or tabs** — accessible but not intrusive.

Surface more internal detail only when:

- Stakes are high and the user needs to verify
- Something has failed and recovery context matters
- The user explicitly asks to *"see what happened"*

The Risks of Getting This Wrong

Too Much Exposure

The interface becomes a **debugging console**, not a product. Users feel overwhelmed and disengage.

Too Little Exposure

Users can't **trust or correct** the system. Opaque agents fail silently — and users blame the product.

📌 Generative UI lets you find the **right middle ground** — structured output makes selective exposure straightforward.

How Is a Basic Chainlit App Structured?

Strip a Chainlit app to its essentials and you get a small Python module with **event handlers** for chat lifecycle events. This is where your agent orchestration layer plugs in.

```
import chainlit as cl

@cl.on_message
async def main(message: cl.Message):
    # Call your Orchestrator here
    # Then decide: text reply or structured UI?
    await cl.Message(
        content=f"Received: {message.content}",
    ).send()
```

Inside `main()`, you invoke your Orchestrator, Support, and Sales agents — then decide what to send back as plain text, a structured card, or an interactive element.

@cl.on_chat_start

Fires when a session begins. Initialize agent state, greet the user, or pre-load context.

@cl.on_message

Fires on every user message. This is your main orchestration entry point.

cl.Message

The primary primitive for sending responses — text, elements, and actions all attach here.

Messages, Steps, and Elements: Mapping UI to Agent Behavior

These three building blocks are how Chainlit translates agent behavior into visible, interactive UI.



Messages

Chat bubbles carrying text and attached elements. The primary output surface for every agent turn.



Steps

Expandable "reasoning blocks" that expose tool calls, LLM calls, or sub-agent work as a **visual trace** — collapsible by default for cleanliness.



Elements

Rich content — tables, images, code blocks, files — attached to messages or steps to build **structured UI cards and panels** from tool results.

```
@cl.step(type="tool", name="Search order system")
async def search_order(order_id: str):
    return {"order_id": order_id, "status": "Shipped"}

@cl.on_message
async def main(message: cl.Message):
    result = await search_order(message.content)
    await cl.Message(
        content=f"Order {result['order_id']} is {result['status']}."
    ).send()
```

Actions and Sessions: Control and Continuity Across Turns

Actions: Explicit Control Surfaces

Actions are **interactive buttons** attached to messages. Clicking them triggers Python callbacks on the backend — giving users direct, structured ways to steer the agent instead of relying solely on free-text prompts.

```
@cl.on_chat_start
async def start():
    await cl.Message(
        content="Check your latest order?",
        actions=[
            cl.Action(name="check_order",
                      value="yes", label="Yes"),
            cl.Action(name="skip",
                      value="no", label="No"),
        ],
    ).send()

@cl.action_callback("check_order")
async def on_check_order(action: cl.Action):
    cl.user_session.set(
        "wants_order_check", True)
    await cl.Message(
        content="Looking up your order..."
    ).send()
```

Sessions: Per-User State Across Turns

User sessions let you store per-user context — which agent handled the last turn, which order ID is active, what preferences were set — so the UI behaves consistently as the Orchestrator and specialists collaborate across a multi-turn journey.

- ✓ Actions give users **explicit control surfaces**. Sessions give your UI **memory and continuity**. Together, they make orchestrated agents feel coherent rather than stateless.

Use Actions for

Confirmations, branching choices, escalation triggers, and any decision with real-world consequences.

Use Sessions for

Tracking active agent, last tool result, user preferences, and handoff metadata across turns.

Key Takeaways

01

Generative UI is an action space

Agents don't just reply — they shape the interface. Design for **structured output**, not just text generation.

02

Transparency + Control are non-negotiable

Show what agents are doing. Give users ways to steer, pause, and recover. Trust is earned through visibility.

03

Separate framework from agent responsibilities

The agent owns the **UI contract**; the framework renders it. Stable structured output enables independent evolution of both layers.

04

Progressive disclosure is your UX policy

Keep the surface clean. Expose traces and internals contextually — never by default, always on demand.

05

Build for journeys, not turns

Group flows, surface summaries, and create foundations for undo and replay. Real tasks are multi-step.